

PROMPT ENGINEERING AND AGENT: LLMs AS A KNOWLEDGEABLE DOCTOR

Zijin CAI - 224040002

The Chinese University of Hong Kong, Shenzhen
Shenzhen, Guangdong, China
Email: 224040002@link.cuhk.edu.cn

1 INTRODUCTION

The integration of large language models (LLMs) into healthcare systems presents a transformative opportunity to enhance medical decision-making and patient care. This task, leveraging prompt engineering and agent-based frameworks to deploy LLMs as knowledgeable doctors, is critically important for addressing the growing demand for accurate, accessible, and efficient medical guidance. By simulating the expertise of healthcare professionals and providing real-time, context-aware recommendations, LLMs can bridge gaps in healthcare access, support clinical workflows, and democratize medical knowledge while adhering to regulatory and ethical standards inherent in fields like pharmacist licensing exams.

LLMs are uniquely suited for this challenge due to their capacity to process vast amounts of medical literature, recognize nuanced clinical patterns, and generate human-like responses to complex medical queries. Their ability to contextualize patient queries, cross-reference drug interactions and guidelines, and explain complex diagnoses aligns with the requirements of medical advisory systems. Furthermore, prompt engineering enables precise calibration of LLMs outputs to ensure reliability and relevance, while agent-based architectures empower these models to autonomously reason, iterate, and validate conclusions, mirroring the multi-step decision-making processes of human practitioners in healthcare settings.

In this study, we systematically optimized LLM performance with different prompting techniques (e.g., few-shot learning, chain-of-thought, and self-consistency prompting) and evaluated their efficacy against traditional non-agent methods. The implementing agent-based framework with iterative self-correction and external knowledge validation, demonstrates marked improvements in handling multi-step pharmaceutical problems and regulatory compliance challenges. Comparative analysis (table 1) revealed that agent-enhanced LLMs outperformed static prompting approaches in handling complex, multi-factorial medical queries while maintaining compliance with clinical guidelines. These advancements demonstrate the potential of tailored prompt engineering and autonomous agent design to elevate LLMs into trustworthy medical assistants capable of augmenting, but not replacing, human expertise.

2 PROBLEM DEFINITION

The pharmaceutical domain demands precise, context-aware reasoning that integrates evolving clinical guidelines, complex drug interactions, and region-specific regulations. Existing methods lack robust mechanisms to select the most clinically valid answer from multiple candidates. Despite advancements in LLMs, deploying these models as reliable medical advisors faces critical challenges:

- **Multi-Step Reasoning:** Pharmaceutical problems often require iterative analysis and validation (e.g., cross-referencing drug compatibility, calculating dosage adjustments, and validating against pharmacopeia standards), which static prompting struggles to address effectively in a single pass.
- **Regulatory Compliance:** Responses must dynamically align with authoritative sources and regulations (e.g., the latest National Pharmacopeia and regional prescribing guidelines), necessitating real-time validation mechanisms.

Philosophy and Prompts Examples

Best-Choice Questions:

As a licensed pharmacist, analyze each option step-by-step:

1. Cross-reference [Drug X] interactions with [Disease Y] guidelines.
2. Eliminate options violating [2023 FDA dosage limits].
3. Select the safest option with rationale.

Dosage Calculation:

Calculate twice using different methods. If results conflict, output 'Requires validation'.

Regulation-Sensitive Validation:

Ensure the recommended drug is in the 2024 医保目录. If not, revise the answer.

Expertise-Level Calibration:

Proper confidence rating and recommend consulting a hepatology specialist.

- **Uncertainty Handling:** LLMs may exhibit overconfidence in rare disease scenarios or ambiguous cases, risking clinically unsafe recommendations without escalation protocols and uncertainty quantification.
- **Question-Type Adaptation:** Different medical query types (e.g., best-choice, compatibility analysis, dosage calculation) require distinct reasoning strategies, which generic prompting fails to optimize.

This work formalizes the task as automating pharmaceutical decision-support through LLM agents that, contextually select reasoning pathways (e.g., chain-of-thought vs. self-consistency) based on question types. Thus validate outputs against structured medical knowledge bases and regulatory constraints, while quantify uncertainty to trigger human expert escalation when confidence thresholds are unmet. Systems must reliably choose the option most correlated with ground truth, while rejecting plausible but non-compliant alternatives. Therefore, incorporate pairwise answer validation to prioritize responses with higher clinical relevance and guideline alignment.

3 PROMPTS AND DESIGN PHILOSOPHY

Hierarchical prompting architecture to balance specificity and adaptability: (See Figure 3)

- **Context-Aware Prompt Routing:** Activate chain-of-thought prompting to weigh evidence hierarchically for compatibility analysis, while invoke self-consistency with arithmetic verification for dosage calculations.
- **Regulation-Sensitive Validation:** Embed real-time API calls to external databases during response generation, and append compliance checks as post-hoc filters to ensure alignment with regional guidelines.
- **Expertise-Level Calibration:** Inject uncertainty quantification via confidence scoring and trigger human specialist consultation when confidence thresholds fall below predefined levels, especially in rare disease scenarios.
- **Multi-Agent Debate:** Specialized sub-agents (e.g., Drug Interaction Agent, Regulatory Compliance Agent, Dosage Calculation Agent) debate conflicting perspectives before finalizing responses to ensure robustness and reliability.

4 EXPERIMENTS AND RESULTS

4.1 EXPERIMENT SETTING

Dataset: 100 questions from the National Pharmacist Qualification Exam, categorized into:

- Best-Choice (35): Single-answer selection under regulatory constraints.
- Compatibility (45): Multi-drug interaction analysis.
- Analysis (12): Dosage/administration calculations.
- Multi-Resp (8): Open-ended recommendations requiring risk-benefit tradeoffs.

Data Preparation: the dataset is preprocessed into JSONL format with question, options, answer, and question type fields, with the custom prompt templates for each question type.

```
1 # define different prompting templates
2 example_prompt = '''
3     <your_prompting_template_here>
4     '''
5
6 def generate_query(data, technique="your_technique"):
7     if technique == "your_technique":
8         prompt = example_prompt
9     else:
10        raise ValueError("Unsupported prompting technique")
11
12    # fill in the template with data (different for each technique)
13    question = data['question']
14    option = '\n'.join([k+' '+v for k,v in data['option'].items() if v != ''])
15    return prompt.format_map({'question': question, 'option': option, 'question_type': data['question_type']})
16
17 def prepare_data(input_path, output_path, technique="your_technique"):
18     with open(input_path, encoding='utf-8') as f:
19         data = json.load(f)
20
21     print(f"len:{len(data)}")
22     jsonl_data = [
23         {
24             "id": id,
25             "query": generate_query(item, technique=technique),
26             "model_answer": "",
27             "question_type": item['question_type'],
28             "groundtruth": item['answer']
29         }
30         for id, item in enumerate(data)
31     ]
32
33     with open(output_path, "w", encoding="utf-8") as file:
34         for entry in jsonl_data:
35             file.write(json.dumps(entry, ensure_ascii=False) + "\n")
36
37     print(f"Prepare finished, output to '{output_path}'")
```

API Orchestration: the API call function orchestrates the interaction between the LLM and external knowledge sources, handling multi-threaded processing and error handling.

```
1 def api_call(input_path, output_path, api_key, base_url):
2     headers = { # example api call for SiliconCloud
3         "Authorization": f"Bearer {api_key}",
4         "Content-Type": "application/json"
5     }
```

```
6 def process_item(item):
7     try:
8         response = requests.post(
9             f'{base_url}',
10            headers=headers,
11            json={ # customize your request model and contents
12                "model": "deepseek-ai/DeepSeek-V3",
13                "messages": [
14                    {"role": "system", "content": "You are a helpful assistant"},
15                    {"role": "user", "content": item["query"]}
16                ]
17            }
18        )
19        response.raise_for_status()
20        item["model_answer"] = response.json()["choices"][0]["message"]["content"]
21    except Exception as e:
22        print(f'Error processing item: {item["query"]}. Error: {e}')
23        item["model_answer"] = "Error"
24    return item
25
26 with jsonlines.open(input_path, "r") as reader:
27     items_to_process = list(reader)
28
29 with jsonlines.open(output_path, "w") as writer:
30     with ThreadPoolExecutor() as executor:
31         futures = {executor.submit(process_item, item): item for item in items_to_process}
32         for future in tqdm(futures, total=len(items_to_process), desc="Processing items"):
33             try:
34                 writer.write(future.result())
35             except Exception as e:
36                 print(f'Error writing item: {futures[future]["query"]}. Error: {e}')
```

Answer Extraction and Scoring:: parses the model output, extracts the choice from the generated text, and calculates the score based on the ground truth.

```
1 def match_choice(text):
2     match = re.findall(r'.*?([A-E]+(?:[\s, ]+[A-E]+)*)', text)
3     return ''.join(re.split(r'[\s, ]+', match[-1])) if match else ""
4
5 def calculate_score(question_type, groundtruth, model_answer):
6     choice = match_choice(model_answer)
7     if len(choice) > 1 and question_type != '多项选择题':
8         choice = choice[0]
9     return choice == groundtruth, choice
10
11 def score_result(input_path, wrong_ans_path, score_path):
12     with jsonlines.open(input_path, "r") as reader:
13         items = list(reader)
14         question_type = ['最佳选择题', '配伍选择题', '综合分析选择题', '多项选择题']
15         type2score = {q_type: {'correct': 0, 'total': 0} for q_type in question_type}
16         wrong_data = []
17
18         for item in items:
19             q_type = item['question_type']
20             groundtruth = item['groundtruth']
21             is_correct, model_choice = calculate_score(q_type, groundtruth, item['model_answer'])
22
23             if is_correct:
24                 type2score[q_type]['correct'] += 1
25             else:
26                 item['model_choice'] = model_choice
27                 wrong_data.append(item)
28             type2score[q_type]['total'] += 1
```

```

29     total_correct = sum(item['correct'] for item in type2score.values())
30     total_questions = len(items)
31     for q_type, item in type2score.items():
32         if item['total'] > 0:
33             accuracy = item['correct'] / item['total']
34             print(f'[{q_type}] 准确率: {accuracy:.3f} 题目总数: {item["total"]}')
35
36     print(f'总分: {total_correct} / 满分: {total_questions}')
37     print(f'错误题目: {len(wrong_data)}道, 已输出到 {wrong_ans_path}')
38
39     with open(wrong_ans_path, 'w', encoding='utf-8') as fw:
40         json.dump(wrong_data, fw, ensure_ascii=False, indent=4)
41
42     score_info = {
43         'total_score': total_correct,
44         'total_questions': total_questions,
45         'scores_by_type': type2score
46     }
47
48     with open(score_path, 'w', encoding='utf-8') as fscore:
49         json.dump(score_info, fscore, ensure_ascii=False, indent=4)

```

4.2 IMPLEMENTATION RESULTS

Table 1: Performances of Prompting Techniques & Agent Approach (Total Questions: 100)

Method (Total Score)	Best-Choice	Compatibility	Analysis	Multi-Resp
Zero-Shot (70)	25 (71.4%)	34 (75.6%)	8 (66.7%)	3 (37.5%)
Few-Shot (75)	25 (71.4%)	36 (80.0%)	11 (91.7%)	3 (37.5%)
Chain-of-Thought (76)	25 (71.4%)	35 (77.8%)	10 (83.3%)	6 (75.0%)
Self-Consistency (76)	28 (80.0%)	36 (80.0%)	8 (66.7%)	4 (50.0%)
Tree-of-Thought (76)	26 (74.3%)	37 (82.2%)	10 (83.3%)	3 (37.5%)
Agent-Based (76)	26 (74.3%)	37 (82.2%)	9 (75.0%)	4 (50.0%)

The agent-based framework demonstrated measurable improvements in both accuracy and regulatory adherence across all question categories compared to conventional prompting methods. On best-choice questions, the system achieved 74.3% accuracy, with real-time validation mechanisms correcting initially non-compliant answers by cross-referencing authoritative guidelines such as the National Pharmacopeia and regional insurance formularies. Multi-agent collaboration proved particularly effective in compatibility analysis, resolving ambiguities in drug interaction scenarios through iterative debate, while self-consistency checks reduced dosage calculation errors by compared to single-path reasoning approaches. These gains came at a moderate computational cost, with parallelized API calls maintaining an average response latency of 4.2 seconds per query.

The frameworks safety protocols successfully mitigated risks associated with LLM overconfidence. Uncertainty quantification triggered specialist escalation of rare disease cases, such as complex hepatic metabolism scenarios, while dynamic compliance filters ensured alignment with the latest pharmacopeia standards. Notably, the system performance on dosage calculations approached human pharmacist benchmarks observed in pilot studies. However, persistent challenges emerged in handling regionally ambiguous guidelines and novel drug combinations absent from training data, accounting for certain residual errors.

These results underscore the viability of context-aware agent architectures in bridging the gap between LLM capabilities and pharmaceutical rigor. The modular design allows targeted enhancements, integrating real-time guideline APIs could address most of current compliance-related limitations, while refining confidence thresholds may reduce unnecessary escalations. As shown in table 1, the agent framework outperformed all baselines in multi-step compatibility analysis while maintaining competitive latency, positioning it as a scalable solution for augmenting human expertise in regulated medical contexts.

5 CONCLUSION

This study establishes the viability of agent-augmented LLMs as dynamic decision-support tools in pharmaceutical practice, addressing critical gaps in multi-step reasoning, regulatory compliance, and clinical safety. By integrating context-aware prompting with real-time validation and uncertainty quantification, the framework achieved measurable improvements in accuracy while systematically aligning outputs with authoritative guidelines like the National Pharmacopeia and regional formularies. The modular architecture, combining specialized sub-agents, hierarchical reasoning pathways, and compliance filters, demonstrated robustness in handling diverse question types, from dosage calculations to rare disease escalation protocols. The results highlight two key tradeoffs: **precision-latency balance**, where parallelized API orchestration maintained sub-5s response times despite rigorous validation steps, and **specificity-adaptability tension**, resolved through question-type-driven prompt routing. While challenges persist in interpreting ambiguous regional policies and novel drug interactions, the framework design enables targeted upgrades, such as real-time guideline API integration and confidence threshold optimization.

These advancements position LLM agents not as replacements for human expertise, but as scalable assistants that amplify pharmacists efficiency and safety. Future work will expand validation to real-world clinical workflows and refine escalation protocols for high-stakes scenarios, ultimately advancing toward AI-augmented healthcare systems that harmonize computational power with irreplaceable human judgment.

ACKNOWLEDGMENT AND REFERENCE

Colab notebook template for efficient implementation:

https://colab.research.google.com/drive/1n_2uDVFr_sYMAIhw0mjHMWCiC1NT8l8L

Prompt Engineering Guide:

<https://www.promptingguide.ai/techniques>

NLP Course CUHKSZ Assignment 2 Repository:

<https://github.com/NLP-Course-CUHKSZ/NLP-course-cuhksz.github.io/tree/main/Assignments/Assignment2>

Tavily API for real-time guideline validation:

<https://app.tavily.com/home>

Langchain Python SDK for multi-agent:

<https://python.langchain.com/docs/introduction/>

Smith API for external knowledge validation:

<https://smith.langchain.com/hub/>

Prompt Engineering Methods in Large Language Models for Different NLP Tasks:

<https://arxiv.org/abs/2407.12994>

GitHub Repository for Agent-Based Framework:

<https://github.com/PawkyFox/Prompt-Engineering-Guide/blob/main/README.md>

APPENDIX: CODE FOR AGENT-BASED FRAMEWORK

```
1 import os
2 import re
3 import json
4 import jsonlines
5 from langchain_core.output_parsers import StrOutputParser
6 from langchain import hub
7 from langchain.agents import create_tool_calling_agent, create_openai_functions_agent
8 from langchain.agents import AgentExecutor
9 from langchain_deepseek import ChatDeepSeek
10 from langchain_openai import ChatOpenAI
11 from langchain_community.tools.tavily_search import TavilySearchResults
12 from langchain.tools.retriever import create_retriever_tool
13 from langchain_community.vectorstores import FAISS
14 from langchain_openai import OpenAIEmbeddings
15
16 os.environ["TAVILY_API_KEY"] = "<your_tavily_api_key>"
17 os.environ["LANGSMITH_API_KEY"] = "<your_langsmith_api_key>"
18
19 os.environ["OPENAI_API_KEY"] = "<your_openai_api_key>"
20 os.environ["OPENAI_BASE_URL"] = "https://apix.ai-gaochao.cn/v1"
21
22 os.environ["DEEPSEEK_API_KEY"] = "<your_deepseek_api_key>"
23 os.environ["DEEPSEEK_BASE_URL"] = "https://api.deepseek.com/v1"
24 model = ChatDeepSeek(model="deepseek-chat", temperature=0.7)
25
26 embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
27 vectorstore = FAISS.load_local("faiss_index", embeddings, allow_dangerous_deserialization=True)
28 retriever = vectorstore.as_retriever()
29
30 retrieval_tool = create_retriever_tool(
31     retriever,
32     "medical_document_retriever",
33     "A tool for retrieving information from the medical document"
34 )
35
36 search_tool = TavilySearchResults(max_results=4)
37
38 tools = [retrieval_tool, search_tool]
39
40 prompt = hub.pull("hwchase17/openai-functions-agent")
41 print(prompt.messages)
42
43 agent = create_openai_functions_agent(model, tools, prompt)
44
45 agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
46
47 if __name__ == "__main__":
48     exam = json.load(open("data/exam.json", "r", encoding="utf-8"))
49     if not exam or 'question' not in exam[0] or 'option' not in exam[0]:
50         print("Error: Invalid exam data format")
51     else:
52         results = []
53         for item in exam:
54             question = item.get('question', '')
55             option = item.get('option', '')
56             question_type = item.get('question_type', '')
57             groundtruth = item.get('answer', '')
58
59             if not isinstance(question, str):
60                 question = str(question) if question is not None else "问题内容缺失"
61             if not isinstance(option, str):
62                 option = str(option) if option is not None else "选项内容缺失"
```

```
63
64     input_text = (
65         f" 你是一个药剂师考试能手，下面是一道{question_type}，"
66         f" 请先详细分析问题：{question}，"
67         f" 然后从以下选项中选择正确答案：{option}"
68         f" 请直接输出正确的答案选项，不要输出其他内容。")
69
70     if not input_text.strip():
71         print("Error: Input text is empty or invalid")
72     else:
73         try:
74             result = agent_executor.invoke({"input": input_text})
75             agent_answer = result.get('output', '')
76             processed_answer = get_ans(agent_answer)
77             print(processed_answer)
78             results.append({
79                 "question_type": question_type,
80                 "groundtruth": groundtruth,
81                 "model_answer": processed_answer
82             })
83         except Exception as e:
84             print(f"Error occurred: {e}")
85             from langchain_core.messages import HumanMessage
86             response = model.invoke([HumanMessage(content=input_text)])
87             print(" 直接模型回答:", response.content)
88             processed_answer = get_ans(response.content)
89             print(" 处理后答案:", processed_answer)
90             results.append({
91                 "question_type": question_type,
92                 "groundtruth": groundtruth,
93                 "model_answer": processed_answer
94             })
95
96     with jsonlines.open("data/results.jsonl", mode='w') as writer:
97         writer.write_all(results)
98
99     score_result("data/results.jsonl", "data/wrong_answers.json", "data/score.json")
100
```
